

Candidate-Set Restriction for Computationally Efficient Decentralized Multi-Robot Task Allocation

James Lott

Student Member, IEEE

*Shiley Marcos School of Engineering
University of San Diego*

San Diego, United States of America
jlott@sandiego.edu

Vahraz Honary

Senior Member, IEEE

*Shiley Marcos School of Engineering
University of San Diego*

San Diego, United States of America
vhonary@sandiego.edu

Abstract—Decentralized multi-robot task-allocation (MRTA) algorithms repeatedly evaluate available tasks, creating computational costs that can constrain execution on embedded robotic hardware. This work evaluates whether a common candidate-set restriction can reduce this cost without substantially degrading team performance. Six decentralized allocators—CBAA, ACBBA, PI, HIPC, DMCHBA, and DGA—are evaluated in a Collaborative Visit mission in which four robots must visit 50 known targets. Across 4,800 paired simulation trials, candidate retention is varied from the complete task set to a single candidate. At 10% retention, mean computation per allocator call decreased by 62.2% across the six methods while mean maximum-robot-step degradation was 3.9%. Individual computational reductions ranged from 25.1% to 88.3% and were strongly allocator dependent. RP2040 hardware-in-the-loop measurements further demonstrate that restriction can change whether computationally intensive allocators satisfy an imposed execution-time limit. These results show that restricting the candidate set can expose useful performance-computation operating regions in decentralized MRTA, but that the appropriate restriction level depends on the allocator.

Index Terms—multi-robot task allocation, decentralized robotics, candidate restriction, embedded computing, computational scalability

I. INTRODUCTION

Multi-robot task allocation (MRTA) distributes tasks among autonomous robots to achieve a team objective. Decentralized approaches avoid a central decision point and provide attractive scalability and fault-tolerance properties, but allocation must instead be repeatedly executed by individual robots as mission state changes [1]. This computation is often evaluated only indirectly despite the increasing use of resource-constrained embedded processors for robotic autonomy.

One approach to computational scalability is to reduce the portion of the assignment problem considered during each allocation. Prior methods have sparsified robot-task assignment matrices or decomposed larger assignment problems [3]. In this study, we instead examine a simple allocator-independent mechanism: before allocation, each robot admits only a ranked subset of its currently valid tasks. The internal allocator is not modified.

We evaluate this mechanism across six decentralized allocators with substantially different decision structures. Rather

than proposing a single optimal candidate limit, the objective is to characterize how candidate-set size changes both mission performance and measured computation. The study asks whether a large fraction of the task set can be removed before the resulting mission penalty outweighs the computational benefit.

II. METHODOLOGY

Experiments use the Collaborative Visit (CV) mission previously employed for comparison of decentralized allocation methods [2]. Four robots operate on a 19×19 grid containing 50 known target locations. All targets must be visited by the team. Motion uses four-connected grid transitions and A* path planning, while allocation and communication are performed asynchronously.

Six decentralized allocators are evaluated: the Consensus-Based Auction Algorithm (CBAA), Asynchronous Consensus-Based Bundle Algorithm (ACBBA), Performance Impact (PI), Hybrid Information and Plan Consensus (HIPC), Distributed Matching-by-Clone Hungarian-Based Algorithm (DMCHBA) [4], and a Decentralized Genetic Algorithm (DGA) [5]. These methods span single-task auctions, bundle and path construction, implicit team planning, deterministic assignment, and evolutionary optimization.

At an allocation event, robot i forms its locally valid candidate set $\mathcal{C}_i$. Because all active CV targets have equal task value, candidates are ranked primarily by Manhattan distance from the robot. Only the first K targets are passed to the allocator. If fewer than K valid targets remain, the complete valid set is retained. Candidate limits of

$$K \in \{50, 38, 25, 13, 5, 3, 2, 1\}$$

correspond to progressively stronger restriction from 100% of the initial 50-target set to a single candidate.

The same 100 scenarios are reused for every algorithm and candidate limit, producing $6 \times 8 \times 100 = 4,800$ completed simulation trials. Mission performance is measured using the maximum number of movement steps completed by any robot, $S_{\max}$, which approximates motion-based mission completion time under constant movement speed. Computational cost is

TABLE I
 PAIRED MEAN CHANGE AT 10% CANDIDATE RETENTION RELATIVE TO
 UNRESTRICTED EXECUTION.

Allocator	$\Delta S_{\max}$	ΔT per call
CBA	0.0%	-25.1%
ACBBA	+1.1%	-50.8%
PI	+3.4%	-43.1%
HIPC	+8.1%	-86.2%
DMCHBA	+8.5%	-88.3%
DGA	+2.1%	-79.7%
Mean	+3.9%	-62.2%

Note: Positive $\Delta S_{\max}$ indicates additional maximum robot steps; negative ΔT indicates reduced computation.

the combined candidate-filtering and allocator execution time per call measured on an NVIDIA Jetson AGX Orin.

A separate hardware-in-the-loop implementation executes candidate restriction and allocation on the RP2040 processor used by the physical robot platform, while mission dynamics remain on the host. A 30-s per-call guard is used to identify conditions that exceed the imposed execution limit.

III. RESULTS

Candidate restriction produced a substantial computational benefit before large mission penalties appeared for several allocators. Table I reports the paired mean change at 10% retention ($K = 5$) relative to each allocator’s unrestricted condition.

The response differs substantially by allocator. CBA experiences no mean maximum-step penalty at 10% retention, while ACBBA, PI, and DGA remain within 3.4% of unrestricted performance. At the same setting, the computationally heavier HIPC, DMCHBA, and DGA obtain approximately 80–88% reductions in per-call computation. HIPC and DMCHBA are more sensitive in mission performance, with maximum-step penalties of approximately 8%.

The RP2040 results demonstrate that these reductions can affect more than relative execution speed. DMCHBA exceeds the imposed execution limit at large candidate sets but becomes feasible at 50% retention and reaches approximately 96 ms per call at 10% retention. Thus, candidate restriction can move an allocator between hardware execution regimes rather than simply providing incremental acceleration.

The complete sweep also shows that stronger restriction is not uniformly better. Performance responses differ among algorithms and can become non-monotonic at very small candidate sets. Consequently, neither a single retained percentage nor a single absolute value of K is appropriate across all allocators.

IV. CONCLUSION

A common candidate-set restriction can substantially reduce decentralized MRTA computation while preserving much of the unrestricted mission performance. In the tested Collaborative Visit problem, retaining only 10% of the initial task set reduced mean allocator computation by 62.2% across six

algorithms while increasing maximum robot steps by 3.9% on average. However, both the magnitude of the computational benefit and the associated mission penalty depend strongly on allocator structure.

These results support treating candidate-set size as a deployment parameter rather than a universal preprocessing constant. They also motivate future work in which candidate limits are selected adaptively from measured computation and available execution deadlines instead of being fixed before a mission.

REFERENCES

- [1] H. Chakraborty, F. Guerin, E. Leclercq, and D. Lefebvre, “Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art,” *Robotics and Autonomous Systems*, vol. 168, p. 104492, 2023.
- [2] S. Nayak, S. Yeotikar, E. Carrillo, *et al.*, “Experimental comparison of decentralized task allocation algorithms under imperfect communication,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 572–579, 2020.
- [3] L. Liu and D. A. Shell, “Multi-level partitioning and distribution of the assignment problem for large-scale multi-robot task allocation,” in *Robotics: Science and Systems VII*, 2011.
- [4] A. Samiei and L. Sun, “Distributed matching-by-clone Hungarian-based algorithm for task allocation of multiagent systems,” *IEEE Transactions on Robotics*, vol. 40, pp. 851–863, 2024.
- [5] R. Patel, E. Rudnick-Cohen, S. Azarm, M. W. Otte, H. Xu, and J. W. Herrmann, “Decentralized task allocation in multi-agent systems using a decentralized genetic algorithm,” in *Proc. IEEE International Conference on Robotics and Automation*, 2020, pp. 3770–3776.